



BLUE NOVA SAS

BASTION

Boveda Personal de Credenciales

GUION DE VIDEO 3

La aplicacion web en vivo (Pyodide)

David Alexander Benz Zambrano

Logica de Programacion - UIDE - Paralelo 1-CIB-1A

Docente: Msc. Lilian Aman - Quito, Ecuador - 2026

Guion Video 3 - BASTION Web en vivo (Pyodide)

Proyecto: BASTION - Boveda Personal de Credenciales **Autor:** David Alexander Benz Zambrano - Blue Nova SAS **Materia:** Logica de Programacion (Msc. Lilian Aman) - UIDE, 1-CIB-1A **URL en vivo:** <https://bluenovasas.github.io/bastion-uide/webapp/> **Duracion objetivo:** 5 a 7 minutos **Tono:** energetico, demostrativo, primera persona (voz de David)

Nota de produccion: el autor ingresa TODOS los datos de ejemplo en vivo durante la grabacion. Nada esta precargado. Conviene tener el navegador en pantalla completa y el panel "Consola Python" abierto desde el inicio para que se vea como cada accion dispara codigo Python real.

Escena 1 - Apertura e introduccion (0:00 - 0:35)

En pantalla: Pantalla completa del navegador ya cargado en la URL del demo. Se ve el encabezado BASTION con la identidad Blue Nova (azul y cyan sobre fondo navy), la barra lateral de categorias a la izquierda y la consola en la parte inferior. Cursor reposando sobre el logo.

Narracion sugerida: "Hola, soy David Benz. Esta es la version web de BASTION, mi boveda personal de credenciales. Y lo importante aqui no es solo como se ve, sino lo que corre por debajo: este es exactamente el mismo archivo bastion.py que programe en Python, pero ejecutandose dentro del navegador gracias a Pyodide. No hay servidor, no hay nube. El codigo Python real trabaja en tu propia maquina."

Escena 2 - La consola Python: ver lo que pasa por dentro (0:35 - 1:25)

En pantalla: Acercamiento (zoom o resaltado) a la barra inferior "Consola Python - lo que pasa por dentro (Pyodide ejecuta bastion.py)". Se senalan con el cursor las lineas de arranque que ya imprimio la pagina: - `Cargando Pyodide (motor de Python)...` - `Pyodide listo. Importando bastion.py...` - `bastion.py cargado. Funciones reales disponibles:` - `calcular_hash, calcular_fortaleza, generar, enmascarar.`

Narracion sugerida: "Antes de tocar nada, miren esta franja de abajo: es la Consola Python. Aqui no muestro mensajes decorativos, muestro lo que realmente esta pasando por dentro. Vean: 'Cargando Pyodide, el motor de Python', luego 'importando bastion.py', y aqui confirma que ya tiene disponibles mis funciones reales: `calcular_hash`, `calcular_fortaleza`, `generar` y `enmascarar`. Cada cosa que yo haga en la interfaz va a aparecer aqui como la llamada de Python que la produjo. Asi la docente puede comprobar que la logica vive en el codigo, no en el diseno."

Escena 3 - Crear la boveda y el hash SHA-256 (1:25 - 2:25)

En pantalla: El autor hace clic en "Crear boveda". Escribe en vivo una clave maestra (por ejemplo, una frase de ejemplo) y observa el medidor "Fortaleza de la clave maestra". Confirma la creacion. Inmediatamente se senala en la consola la aparicion de:

```
- >>> bastion.calcular_hash("*****") - <<< "07480fb9e85b9396af..." (hash SHA-256)
- guardar_boveda() -> boveda.json escrito en el navegador
```

Narracion sugerida: "Voy a crear mi boveda. Escribo aqui mi clave maestra; fijense que mientras escribo, un medidor calcula la fortaleza en tiempo real. Y ahora lo clave: miren la consola. En el momento de crear la boveda se ejecuto bastion.calcular_hash con mi clave, y me devuelve este hash SHA-256, esta cadena de 64 caracteres. Mi clave maestra nunca se guarda en texto plano: solo se guarda su huella, su hash. Y aqui abajo confirma 'guardar_boveda, boveda.json escrito en el navegador'. Esa es la capa de persistencia trabajando."

Escena 4 - Generar una contraseña con secrets (2:25 - 3:30)

En pantalla: El autor abre "Generar contraseña". Mueve el control de longitud (por ejemplo a 16) y activa las casillas: mayusculas, minusculas, numeros y simbolos.

Aparece la contraseña generada y la etiqueta "Fortaleza (calculada en Python)"

marcando "Fuerte". Se senala en la consola: - >>> generar_contraseña(16, mayus=True, min=True, num=True, simb=True) - <<< "...la clave generada..." -> fortaleza: Fuerte

Opcional: hacer clic en "Regenerar" una o dos veces para mostrar que cambia, y luego "Usar en credencial".

Narracion sugerida: "Ahora generemos una contraseña segura. Elijo la longitud, digamos dieciseis caracteres, y marco que incluya mayusculas, minusculas, numeros y simbolos. Listo, ahi esta. Y otra vez, vamos a la consola: se llamo a generar_contraseña con esos parametros, y por dentro usa secrets.choice. No uso random comun: uso secrets, que es aleatoriedad criptograficamente segura, la apropiada para seguridad. Y la fortaleza, este 'Fuerte', no lo decide el navegador: lo calcula mi funcion calcular_fortaleza en Python, segun la longitud y cuantos tipos de caracteres incluí. Si regenero, cambia, porque cada clave es nueva. Esta me gusta, la voy a usar en una credencial."

Escena 5 - Agregar credenciales y llenar el sidebar (3:30 - 4:45)

En pantalla: El autor usa "Agregar credencial" varias veces, llenando en vivo nombre, usuario, contraseña y eligiendo categoria del selector. Sugerido agregar cinco, una por categoria: 1. Instagram - categoria Redes sociales 2. Banco Pichincha - categoria Banca y finanzas 3. GitHub Bluenovasas - categoria Claves API / SSH (o Trabajo) 4. Gmail - categoria Correo 5. Servidor SSH (n8n) - categoria Claves API / SSH

Mientras agrega, se ve el sidebar de categorias incrementar su contador y las estadísticas actualizarse. En la consola se senala, por cada alta: - >>>

```
boveda["credenciales"]["Instagram"] = {categoria, usuario, contrasena} -
<<< credencial "Instagram" guardada en el diccionario
```

Narracion sugerida: "Vamos a llenar la boveda. Agrego mi Instagram, en Redes sociales. Vean la consola: se ejecuta la asignacion al diccionario de credenciales, exactamente como en mi codigo, y confirma 'credencial guardada en el diccionario'. Sigo: mi Banco Pichincha en Banca y finanzas. Mi GitHub de Bluenovasas en Claves API y SSH. Mi Gmail en Correo. Y mi servidor por SSH, tambien en Claves API y SSH. Miren como la barra lateral de categorias se va llenando, cada categoria muestra cuantas credenciales tiene, y las estadísticas de aqui arriba se actualizan solas. Esa es la tupla de categorias y el diccionario de la Unidad cuatro, vivos."

Escena 6 - Buscar, revelar y copiar (4:45 - 5:35)

En pantalla: El autor escribe en el buscador (por ejemplo "git" o "gmail"). La lista se filtra al instante. Abre la tarjeta de esa credencial: la contrasena aparece enmascarada (por ejemplo B*****a). Hace clic en el icono de revelar (ojo) para mostrarla completa y luego en copiar; aparece el aviso de copiado.

Narracion sugerida: "Cuando ya tengo varias, necesito encontrarlas rapido. Escribo en el buscador y la lista se filtra al instante, sin distinguir mayusculas. Abro esta credencial: la contrasena se muestra enmascarada, solo el primer y el ultimo caracter, eso lo hace mi funcion enmascarar en Python. Si la necesito de verdad, hago clic en el ojo para revelarla, y aqui la copio al portapapeles con un clic. Comoda y segura: por defecto, oculta."

Escena 7 - Historial de accesos y archivo de la boveda (5:35 - 6:25)

En pantalla: El autor abre "Historial de accesos". Se muestra la lista de eventos con fecha y hora. Se senala en la consola la linea con sabor academico: - >>> for i, r in enumerate(boveda['historial']): # LISTA (Unidad 4) - <<< N accesos registrados

Luego cierra el modal y abre "Ver archivo de la boveda", que muestra el contenido JSON (hash_maestra, credenciales, historial) tal como se guarda en el navegador.

Narracion sugerida: "Aqui esta el Historial de accesos, que es la lista de la Unidad cuatro. Cada vez que inicio sesion, se registra un evento con su fecha y hora. Vean la consola: recorre el historial con un for y enumerate sobre la lista, igualito al codigo. Y por transparencia total, abro 'Ver archivo de la boveda': este es el JSON real que se guarda, con el hash de la maestra, mis credenciales y el historial. Nada escondido: pueden ver como se estructura la informacion por dentro."

Escena 8 - Las tres capas y la invitacion a probarla (6:25 - 6:55)

En pantalla: Vista general de la pantalla. Sugerido un grafico o rotulo sencillo en pantalla con las tres capas: Presentacion (la web) / Logica (bastion.py via Pyodide) / Persistencia (JSON en el navegador). Se muestra grande la URL del demo.

Narracion sugerida: "Recapitulando la arquitectura: lo que vieron son tres capas. La presentacion, esta interfaz web. La logica, mi archivo bastion.py corriendo con Pyodide, que es la misma que usa la version de consola. Y la persistencia, el archivo JSON guardado localmente. Y lo mejor: esto esta en vivo. Profesora, usted puede entrar a esta direccion desde cualquier navegador y probar BASTION usted misma, crear su boveda y ver la consola Python reaccionar."

Escena 9 - Cierre (6:55 - 7:00)

En pantalla: Cierre sobre el logo BASTION con la identidad Blue Nova. Rotulo final con autor, materia y URL del demo.

Narracion sugerida: "Eso es BASTION: Python real, en el navegador, hecho con la logica que aprendi en las cuatro unidades. Gracias por ver. Soy David Benz, para la UIDE y Blue Nova."

Resumen de escenas

#	Escena	Tiempo aprox.	Foco en consola Python
1	Introduccion (Python real con Pyodide)	0:00 - 0:35	-
2	Abrir consola y explicar el "por dentro"	0:35 - 1:25	Arranque Pyodide / importacion bastion.py
3	Crear boveda + clave maestra	1:25 - 2:25	<code>calcular_hash</code> (SHA-256), <code>guardar_boveda</code>
4	Generar contrasena en el modal	2:25 - 3:30	<code>generar_contrasena</code> + <code>secrets</code> , fortaleza en Python
5	Agregar credenciales por categoria	3:30 - 4:45	Asignacion al diccionario, sidebar y estadisticas
6	Buscar, revelar y copiar	4:45 - 5:35	<code>enmascarar</code>
7	Historial y archivo de la boveda	5:35 - 6:25	<code>for ... enumerate(historial)</code> (Lista Unidad 4), JSON
8	Tres capas + invitacion a probar la URL	6:25 - 6:55	-
9	Cierre	6:55 - 7:00	-

Notas de fidelidad (verificado contra el demo en vivo y el codigo del repo)

- Las **categorias reales** del sidebar/selector web son: **Redes sociales, Banca y finanzas, Trabajo, Correo, Claves API / SSH, Otros** (definidas como TUPLA `CATEGORIAS` en `bastion.py`). El video del enunciado nombra Instagram, Banco, GitHub, Gmail y SSH como *ejemplos de credenciales*, que se asignan a esas categorias (Instagram->Redes sociales, Banco->Banca y finanzas, GitHub->Claves API / SSH o Trabajo, Gmail->Correo, SSH->Claves API / SSH).
- La version web usa `secrets` (no `random`): la linea real es `contrasena_generada += secrets.choice(alfabeto)`.
- Los textos de consola citados en el guion son **literales** del demo (`app.js`): por ejemplo `bastion.py` cargado. Funciones reales disponibles: `>>> bastion.calcular_hash("*****")`, `<<< "...24 chars..."` (hash SHA-256), `guardar_boveda()` -> `boveda.json` escrito en el navegador, `>>> generar_contrasena(16, mayus=True, ...)`, `<<< "..."` -> fortaleza: Fuerte, `>>> boveda["credenciales"]` `["Nombre"] = {categoria, usuario, contrasena}`, y `>>> for i, r in enumerate(boveda['historial']): # LISTA (Unidad 4)`.

- El panel de seguridad de la pagina aclara que la clave maestra se guarda como **hash SHA-256** y que **AES-256-GCM + Argon2id** quedan como evolucion del producto; conviene no afirmar que el contenido ya esta cifrado.